

Supplementary Materials: Structured Schemas for LLM-Modeler Collaboration in Quantitative Systems Pharmacology Model Calibration

Joel Eliason and Aleksander S. Popel

Contents

1	Complete SubmodelTarget Example	3
1.1	Overview	3
1.2	Complete YAML Specification	3
1.3	Schema Structure Summary	7
2	Supported Model Types	8
3	Validation Checks	10
3.1	Summary of Validators	10
3.2	DOI Resolution Validator	10
3.3	Value-in-Snippet Validator	11
3.4	Unit Validation	13
3.5	Reference Consistency Validators	13
3.6	Code Execution Validation	14
3.7	Hardcoded Constant Detection	14
3.8	Scale Consistency Validation	14
3.9	Control Character Validation	14
3.10	CalibrationTarget-Specific Validation	14
4	Generated Julia Code Structure	14
5	Pydantic Model Definitions	15
5.1	Top-Level Model	15
5.2	Input Model	16
5.3	Model Type Discriminated Union	17
5.4	Prior Specification	18
5.5	Loading and Validating YAML Files	19
5.6	CalibrationTarget Model	20
5.7	Observable	21
5.8	CalibrationTargetEstimates	21
5.9	Scenario and Intervention	22

33	6 SubmodelTarget Schema Details	23
34	6.1 Input Fields	23
35	6.2 Parameter and Prior Specification	24
36	6.3 Forward Model Specification	24
37	6.4 Error Model Specification	25
38	7 Complete CalibrationTarget Example	26
39	7.1 Overview	26
40	7.2 Observable	26
41	7.3 Empirical Data	27
42	7.4 Source and Context	29
43	8 SubmodelTarget Source Characteristics	30
44	8.1 Source Quality	30
45	8.2 Species Translation	30
46	8.3 Indication Match	31
47	9 CalibrationTarget Detailed Metrics	31
48	9.1 Observable Categories	31
49	9.2 Observable Complexity	31
50	9.3 Extraction Model Attribution	32
51	9.4 Primary Data Sources	32
52	10 Inference Results	32
53	10.1 Convergence Diagnostics	33
54	10.2 Posterior Distributions	33
55	11 Model-Aware Prompt Construction Details	33
56	12 Schema Implementation Details	36
57	13 Source Relevance Assessment Details	36
58	14 Collaboration Mode Details	37
59	15 Detailed Comparison to Existing Approaches	37

1 Complete SubmodelTarget Example

This section presents a complete SubmodelTarget for activated pancreatic stellate cell (aPSC) proliferation, demonstrating all schema components.

1.1 Overview

This target constrains the proliferation rate parameter `k_apsc_prolif` using data from Schneider et al. (2001), who measured PSC proliferation in response to PDGF stimulation via BrdU incorporation assay.

1.2 Complete YAML Specification

```
# Schema: SubmodelTarget
# Target: Activated PSC proliferation rate

target_id: psc_proliferation_PDAC_deriv001

# =====
# INPUTS: Extracted values with full provenance
# =====
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."
```

```
  - name: pdgf_fold_sd
    value: 0.49
    units: dimensionless
    input_type: direct_measurement
    role: auxiliary
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."
```

```
# =====
# CALIBRATION: Everything needed for inference
# =====
calibration:
  parameters:
    - name: k_apsc_prolif
```

```

103     units: 1/day
104     prior:
105         distribution: lognormal
106         mu: 0.0      # log(1.0) - expect ~1/day for 4x fold-change in 1 day
107         sigma: 1.0
108         rationale: |
109             Wide log-normal prior centered at 1/day. A 4-fold increase in
110             1 day implies  $k = \ln(4) = 1.4/\text{day}$ , so prior median of 1/day
111             is reasonable.
112
113 forward_model:
114     type: exponential_growth
115     rate_constant: k_apsc_prolif
116     data_rationale: |
117         BrdU incorporation assay measures proliferation as fold-change
118         over ~24 hours under constant PDGF stimulus. With a single high
119         PDGF dose and short duration, exponential growth ( $dN/dt = k \cdot N$ )
120         fits the data structure.
121     submodel_rationale: |
122         The full QSPiO_PDAC model has aPSC proliferation via:
123          $k_{\text{apsc\_prolif}} * \text{aPSC} * (\text{PDGF} / (\text{PDGF}_{50\_prolif} + \text{PDGF}))$ .
124         At 50 ng/mL PDGF  $\gg$  PDGF50_prolif, the saturation term  $\sim 1$ ,
125         reducing to exponential growth with rate  $k_{\text{apsc\_prolif}}$ .
126     independent_variable:
127         name: time
128         units: day
129         span: [0.0, 1.0]
130         rationale: |
131             Proliferation assay duration assumed to be ~24 hours based on
132             typical cytokine incubation protocols for rat PSCs.
133     state_variables:
134         - name: aPSC_rel
135           units: dimensionless
136           initial_condition:
137               value: 1.0
138               rationale: |
139                   PSC proliferation data are reported as fold of control. We
140                   normalize the initial aPSC population to 1.0 (100% control)
141                   to match this convention.
142
143 error_model:
144     - name: proliferation_fold_change
145       observable:
146           type: identity
147           state_variables: [aPSC_rel]
148           rationale: |
149               The state variable aPSC_rel is the fold-change in aPSC number
150               relative to baseline (control = 1.0), which directly matches

```

```

151         the reported proliferation fold-change.
152     units: dimensionless
153     uses_inputs:
154         - pdgf_fold_mean
155         - pdgf_fold_sd
156     evaluation_points: [1.0]
157     sample_size: 3
158     sample_size_rationale: |
159         Sample size not explicitly reported; assumed n=3 independent
160         experiments, typical for in vitro PSC BrdU proliferation assays.
161     distribution_code: |
162         def derive_distribution(inputs, ureg):
163             import numpy as np
164
165             rng = np.random.default_rng(42)
166
167             mean = inputs['pdgf_fold_mean'].magnitude
168             sd = inputs['pdgf_fold_sd'].magnitude
169             n = 10000
170
171             # Use lognormal for positive fold-changes
172             mu_log = np.log(mean**2 / np.sqrt(mean**2 + sd**2))
173             sigma_log = np.sqrt(np.log(1 + sd**2 / mean**2))
174
175             samples = rng.lognormal(mu_log, sigma_log, n)
176
177             median_val = np.median(samples)
178             ci_lower = np.percentile(samples, 2.5)
179             ci_upper = np.percentile(samples, 97.5)
180
181             return {
182                 'median': [median_val],
183                 'ci95': [[ci_lower, ci_upper]],
184                 'units': 'dimensionless'
185             }
186     likelihood:
187         distribution: lognormal
188         rationale: |
189             Fold-changes are positive and multiplicative, making
190             lognormal appropriate.
191
192     identifiability_notes: |
193         With a single timepoint (~24h) and normalized initial condition
194         (aPSC_rel(0)=1), only k_apsc_prolif is estimated. If substantial
195         PSC death occurred over the assay interval, the inferred rate
196         would represent net growth rather than pure proliferation.
197
198     # =====

```

```

199 # EXPERIMENTAL CONTEXT
200 # =====
201 experimental_context:
202     species: rat
203     system: in_vitro_primary_cells
204     indication: PDAC
205     cell_types:
206         - name: pancreatic stellate cell
207           phenotype: activated (alpha-SMA positive)
208           isolation_method: Isolated from normal Wistar rat pancreas,
209                           activated by culture on plastic
210     culture_conditions:
211         culture_type: 2d_monolayer
212
213 # =====
214 # STUDY INTERPRETATION
215 # =====
216 study_interpretation: |
217     Schneider et al. (2001) measured proliferation of cultured rat
218     pancreatic stellate cells (PSCs) in response to growth factors using
219     BrdU incorporation. Addition of 50 ng/mL PDGF increased PSC
220     proliferation to 4.37 +/- 0.49-fold of control, indicating that PDGF
221     acts as a strong mitogen for activated PSCs. We interpret the
222     50 ng/mL PDGF condition as approximating a near-maximal PDGF stimulus
223     and use the reported fold-change over ~24 hours as a quantitative
224     constraint on the maximal PDGF-driven proliferation rate parameter
225     k_apsc_prolif.
226
227 key_assumptions:
228     - |
229         50 ng/mL PDGF corresponds to a near-saturating concentration for
230         PSC proliferation, so  $PDGF/(PDGF_{50\_prolif} + PDGF) \sim 1$ .
231     - |
232         The proliferation assay duration is approximately 24 hours,
233         consistent with typical cytokine incubation protocols.
234     - |
235         BrdU incorporation fold-change is proportional to fold-change in
236         PSC cell number (negligible death over 24 hours).
237     - |
238         Rat activated PSC proliferation kinetics in vitro provide an upper
239         bound for human tumor aPSC proliferation in vivo.
240
241 key_study_limitations:
242     - |
243         Proliferation was measured via BrdU incorporation rather than
244         direct cell counts.
245     - |
246         Single PDGF concentration and single assay interval constrain

```

```

247     only the maximal rate.
248 - |
249     Data from rat PSCs in vitro, not human PDAC-associated stellate cells.
250 - |
251     Background serum proliferative signals not explicitly modeled.
252
253 # =====
254 # DATA SOURCES
255 # =====
256 primary_data_source:
257     doi: "10.1152/ajpcell.2001.281.2.C532"
258     title: "Identification of mediators stimulating proliferation and
259           matrix synthesis of rat pancreatic stellate cells"
260     authors: [Schneider, Schmid, Liptay, Schmid, Pohl]
261     year: 2001
262     source_tag: Schneider2001_PSC_PDGF
263
264 secondary_data_sources:
265 - doi: "10.1136/gut.44.4.534"
266   title: "Pancreatic stellate cells are activated by proinflammatory
267         cytokines: implications for pancreatic fibrogenesis"
268   authors: [Apte, Haber, Darby, Rodgers, McCaughan, Korsten, Pirola, Wilson]
269   year: 1999
270   source_tag: Apte1999_PSC_Cytokines
271   contribution: |
272     Provides context on PSC activation and cytokine response protocols.

```

273 1.3 Schema Structure Summary

274 The SubmodelTarget schema has the following top-level structure:

- 275 • **target_id**: Unique identifier for the target
- 276 • **inputs**: List of values extracted from literature with provenance
- 277 • **calibration**: Everything needed for inference code generation
 - 278 - **parameters**: Parameters to estimate with priors
 - 279 - **forward_model**: Forward model specification (built-in type or custom). For ODE mod-
280 els, contains nested **state_variables** (state variables with initial conditions) and **independent_variabl**
281 (e.g. time)
 - 282 - **error_model**: Observable specifications mapping model predictions to data with likeli-
283 hoods
 - 284 - **identifiability_notes**: Discussion of parameter identifiability
- 285 • **experimental_context**: Species, system, cell types, culture conditions
- 286 • **study_interpretation**: Scientific interpretation of the data
- 287 • **key_assumptions**: Assumptions made in using this data

- 288 • **key_study_limitations:** Known limitations of the source study
- 289 • **primary_data_source:** Primary literature reference with DOI
- 290 • **secondary_data_sources:** Additional references

291 2 Supported Model Types

292 The SubmodelTarget schema supports 15 built-in forward model types, organized into four cate-
293 gories.

Table 1: Built-in forward model types organized by category.

Category	Model Type	Formulation	Key Fields
ODE templates	exponential_growth	$dy/dt = k \cdot y$	rate_constant
	first_order_decay	$dy/dt = -k \cdot y$	rate_constant
	logistic	$dy/dt = k \cdot y(1 - y/K)$	rate_constant, carrying_capacity
	saturation	$dy/dt = k(1 - y)$	rate_constant
	two_state	$dA/dt = -kA, dB/dt = kA$	forward_rate
	michaelis_menten	$dy/dt = -V_{\max}y/(K_m + y)$	vmax, km
Structured steady-state	steady_state_density	rate = density $\times$ loss	target_rate, observed_density, loss_rate
	steady_state_fraction	rate = $f \times \rho_{\text{parent}} \times \text{loss}$	target_rate, observed_fraction, parent_density
	steady_state_concentration	sec = $C \times k_{\text{cl}} \times V$	target_rate, observed_concentration, clearance_rate
	steady_state_ratio	$r = k_1/k_2$	target_rate, observed_ratio, opposing_rate
	steady_state_proliferation	$f = k_p/(k_p + k_d)$	target_rate, observed_fraction, death_rate
Accumulation	batch_accumulation	$C = (\text{sec} \times n \times t)/V$	target_rate, cell_count, incubation_time
Generic/ fallback	algebraic	(closed-form formula)	code, code_julia
	direct_fit	(curve fitting)	curve
	custom	(user-provided ODE)	code, code_julia

Each structured steady-state and accumulation model declares its fields as typed roles: a field can reference a calibration parameter (to be estimated), an extracted input (from the inputs layer), a curated reference value from a shared database, or a numeric literal. This role-based design allows the same template to serve different estimation scenarios depending on which field is treated as the unknown.

3 Validation Checks

The validation framework comprises several dozen Pydantic `@model_validator` and `@field_validator` methods spread across the `SubmodelTarget` and `CalibrationTarget` schemas. They run after field parsing and raise a `ValidationError` with detailed messages when constraints are violated. The implementation is granular (many small validators), but the checks fall into the ten categories below.

3.1 Summary of Validators

1. **Input reference validation:** All `uses_inputs` references match defined input names
2. **Source reference validation:** All `source_ref` values match defined source tags
3. **Parameter reference validation:** Model parameter roles reference defined parameters
4. **State variable reference validation:** Observable `state_variables` match defined state variables
5. **DOI resolution:** All DOIs resolve via CrossRef API
6. **Value-in-snippet validation:** Extracted values appear verbatim in quoted snippets
7. **Unit validation:** All unit strings parse with Pint
8. **Code syntax validation:** Custom code passes Python AST parsing
9. **Span ordering validation:** Time spans have `start < end`
10. **ODE model requirements:** ODE models have required state variables and spans

3.2 DOI Resolution Validator

The DOI validator queries the CrossRef API to verify that every cited DOI resolves to a valid publication. This catches fabricated citations before they enter the calibration pipeline. The validator also performs fuzzy matching on title and year to detect metadata inconsistencies.

```
def resolve_doi(doi: str) -> Optional[dict]:
    """Resolve DOI and get metadata from CrossRef."""
    doi_clean = doi.replace("https://doi.org/", "").strip()

    url = f"https://doi.org/{doi_clean}"
    headers = {"Accept": "application/vnd.citationstyles.csl+json"}
    response = requests.get(url, headers=headers, timeout=10)
```

```

329     if response.status_code != 200:
330         return None # DOI does not resolve
331
332     metadata = response.json()
333     return {
334         "title": metadata.get("title", ""),
335         "first_author": metadata.get("author", [{}])[0].get("family", ""),
336         "year": metadata.get("issued", {}).get("date-parts", [[]])[0][0]
337     }
338
339 @model_validator(mode="after")
340 def validate_doi_resolution_and_metadata(self) -> "SubmodelTarget":
341     """Validate that DOIs resolve and metadata matches."""
342     errors = []
343
344     metadata = resolve_doi(self.primary_data_source.doi)
345     if metadata is None:
346         errors.append(f"DOI '{self.primary_data_source.doi}' failed to resolve")
347     else:
348         # Check title match (fuzzy)
349         if not fuzzy_match(self.primary_data_source.title, metadata["title"]):
350             errors.append(f"Title mismatch with CrossRef")
351         # Check year match
352         if self.primary_data_source.year != metadata["year"]:
353             errors.append(f"Year mismatch: {self.primary_data_source.year} vs {metadata['year']}")
354
355     if errors:
356         raise ValueError("DOI validation errors:\n - " + "\n - ".join(errors))
357     return self

```

358 **Example failure:** If an LLM fabricates a citation with DOI 10.1234/fake.2023.12345, the
359 CrossRef query returns a 404 error and validation fails with:

```

360 ValidationError: DOI '10.1234/fake.2023.12345' failed to resolve.
361 Verify at https://doi.org/10.1234/fake.2023.12345

```

362 3.3 Value-in-Snippet Validator

363 The value-in-snippet validator checks that each extracted numeric value appears in its associated
364 source text. This catches hallucinations where an LLM extracts a plausible but incorrect value. The
365 validator handles multiple numeric formats including scientific notation, percentages, and Unicode
366 superscripts.

```

367 def check_value_in_text(text: str, value: float) -> bool:
368     """Check if numeric value appears in text."""
369     text_norm = text.lower().replace(",", "")
370     patterns = []
371
372     # Direct value and common formats

```

```

373     patterns.append(str(value))
374     patterns.append(f"{value:.1f}")
375     patterns.append(f"{value:.2f}")
376
377     # Scientific notation (e.g., "1.5e-3", "1.5E-03")
378     if abs(value) < 0.01 or abs(value) > 1000:
379         patterns.append(f"{value:e}")
380         patterns.append(f"{value:.2e}")
381
382     # Percentage format (if value is between 0 and 1)
383     if 0 < value < 1:
384         pct = value * 100
385         patterns.extend([f"{pct}%", f"{pct:.0f}%", f"{pct:.1f}%"])
386
387     return any(p.lower() in text_norm for p in patterns)
388
389 @model_validator(mode="after")
390 def validate_input_values_in_snippets(self) -> "SubmodelTarget":
391     """Validate that extracted values appear in their value_snippet."""
392     errors = []
393     for inp in self.inputs:
394         if not inp.value_snippet:
395             continue
396         # Skip types that may not have explicit values in text
397         if inp.input_type in (InputType.INFERRED_ESTIMATE, InputType.ASSUMED_VALUE):
398             continue
399
400         if not check_value_in_text(inp.value_snippet, inp.value):
401             errors.append(
402                 f"Input '{inp.name}': value {inp.value} not found in snippet "
403                 f"'{inp.value_snippet[:60]}...'")
404
405     if errors:
406         raise ValueError("Value-snippet mismatches (possible hallucination):\n - "
407                           + "\n - ".join(errors))
408
409     return self

```

410 **Example failure:** If an input claims value: 4.37 but the snippet reads “Cell proliferation
411 was 3.21-fold of control”, validation fails with:

```

412 ValidationError: Value-snippet mismatches (possible hallucination):
413 - Input 'pdgf_fold_mean': value 4.37 not found in snippet
414   'Cell proliferation was 3.21-fold of control...'

```

415 This validator is particularly effective because LLMs that hallucinate values rarely fabricate
416 consistent snippets, so the mismatch provides a reliable detection signal.

3.4 Unit Validation

The unit validator parses all unit strings using Pint, a Python library for physical quantities. Malformed or unrecognized units trigger validation failure.

```
@model_validator(mode="after")
def validate_units_are_valid_pint(self) -> "SubmodelTarget":
    """Validate that all unit strings are valid Pint units."""
    from pint import UnitRegistry
    ureg = UnitRegistry()
    errors = []

    def check_unit(unit_str: str, location: str):
        try:
            ureg(unit_str)
        except Exception as e:
            errors.append(f"{location}: '{unit_str}' is not valid ({e})")

    for inp in self.inputs:
        check_unit(inp.units, f"Input '{inp.name}'")
    for param in self.calibration.parameters:
        check_unit(param.units, f"Parameter '{param.name}'")

    if errors:
        raise ValueError("Invalid Pint units:\n - " + "\n - ".join(errors))
    return self
```

Example failure: If a parameter specifies units: `cells/mL/day` (invalid compound unit), validation fails with:

```
ValidationError: Invalid Pint units:
- Parameter 'k_recruit': 'cells/mL/day' is not valid (...)
```

3.5 Reference Consistency Validators

Four validators check that all cross-references within the schema are consistent:

- **Input references:** Every `uses_inputs` entry and `initial_condition.input_ref` must match a defined input name.
- **Source references:** Every `source_ref` must match `primary_data_source.source_tag` or a `secondary_data_sources[].source_tag`.
- **Parameter references:** When a model field like `rate_constant` contains a string (not an `InputRef`), it must match a parameter name in `calibration.parameters`.
- **State variable references:** Every state variable referenced in `observable.state_variables` must exist in `calibration.state_variables`.

These validators catch internal inconsistencies where an LLM generates references to entities that don't exist elsewhere in the document.

3.6 Code Execution Validation

Code validation goes beyond syntax checking. The validator parses the code structure, then executes it with mock data to verify correct behavior. For `observation_code`, execution must return the required dictionary keys (`value`, `sd`, `sd_uncertain`) with appropriate types. For observable code, execution must return arrays with correct units and dimensions. This catches logic errors and signature mismatches before inference.

3.7 Hardcoded Constant Detection

Hardcoded constant detection scans code blocks for numeric literals that should be documented inputs. The parser identifies constants (e.g., 1440 for a minutes-to-days conversion) that are not in the allowed set (small integers 0–5, common fractions, statistical percentiles). Each flagged constant must either be moved to an input with provenance or documented as an `observable_constant` with a `biological_basis` explanation.

3.8 Scale Consistency Validation

Scale consistency validation executes observable code with mock species data spanning a 1000-fold range. If the output is constant despite varying inputs, or if the output magnitude suggests a ratio/score mismatch (e.g., values near 100 when 0–1 is expected), validation warns of potential unit or scaling errors.

3.9 Control Character Validation

Control character validation scans all string fields for characters that corrupt YAML parsing (ASCII control codes, non-breaking spaces, and similar). PDF copy-paste often introduces invisible characters that cause parsing failures; this validator catches them before they propagate.

3.10 CalibrationTarget-Specific Validation

CalibrationTarget adds validators for full-model observables. Observable code is executed against mock species data to verify correct function signatures, output dimensionality, and unit consistency. A denominator audit requires that density or fraction observables explicitly document what tissue area or cell population serves as the denominator, both experimentally and in the model, catching a common source of order-of-magnitude errors. Species existence checks verify that every species referenced in the observable code exists in the model structure. Distribution code is executed with the declared inputs to verify that computed summary statistics (median and CI95) match the values reported in the empirical data block within tolerance (1% for median, 10% for CI bounds), catching transcription errors and code bugs before they propagate to calibration.

4 Generated Julia Code Structure

The translator generates Julia scripts with the following structure:

```
using DifferentialEquations
using Turing
using Distributions
```

```

494 # Observed data (from inputs layer)
495 const obs_proliferation_median = 4.37
496 const obs_proliferation_sigma = 0.25 # derived from CI95
497
498 # ODE function
499 function ode_proliferation!(du, u, p, t)
500     k = p[1]
501     du[1] = k * u[1] # exponential growth
502 end
503
504 # Simulate function
505 function simulate_proliferation(k; t_span=(0.0, 1.0), u0=[1.0])
506     prob = ODEProblem(ode_proliferation!, u0, t_span, [k])
507     sol = solve(prob, Tsit5())
508     return sol[end][1] # final value
509 end
510
511 # Turing model
512 @model function proliferation_model()
513     # Prior
514     k_apsc_prolif ~ LogNormal(0.0, 1.0)
515
516     # Simulate
517     pred = simulate_proliferation(k_apsc_prolif)
518
519     # Likelihood
520     obs_proliferation_median ~ LogNormal(log(pred), obs_proliferation_sigma)
521 end
522
523 # Run inference
524 model = proliferation_model()
525 chain = sample(model, NUTS(), MCMCThreads(), 1000, 4)

```

526 For joint inference over multiple targets, the translator identifies shared parameters by name
527 and generates a single model with one prior per unique parameter.

528 5 Pydantic Model Definitions

529 This section provides the key Pydantic model definitions that implement both schemas. The full
530 implementations are available in the repository at `src/qsp_llm_workflows/core/calibration/`.

531 5.1 Top-Level Model

532 The `SubmodelTarget` class is the entry point for validation. When a YAML file is loaded, Pydantic
533 recursively validates all nested models.

```

534 class SubmodelTarget(BaseModel):
535     """

```

```

536     Submodel-based calibration target for QSP parameter inference.
537
538     Separates:
539     - 'inputs': What was extracted from papers (with full provenance)
540     - 'calibration': How to use those inputs for inference
541     """
542     target_id: str = Field(description="Unique identifier for this target")
543
544     # Extracted data with provenance
545     inputs: List[Input] = Field(
546         description="Values extracted from literature with full provenance"
547     )
548
549     # Calibration specification
550     calibration: Calibration = Field(
551         description="Everything needed for inference code generation"
552     )
553
554     # Experimental context
555     experimental_context: ExperimentalContext = Field(
556         description="Experimental context for the target"
557     )
558
559     # Study interpretation
560     study_interpretation: str = Field(
561         description="Scientific interpretation of how the study data informs the model"
562     )
563     key_assumptions: List[str] = Field(
564         description="Key assumptions made in using this data"
565     )
566     key_study_limitations: Optional[List[str]] = Field(
567         default=None, description="Known limitations of the study"
568     )
569
570     # Data sources
571     primary_data_source: PrimaryDataSource = Field(
572         description="Primary literature source"
573     )
574     secondary_data_sources: Optional[List[SecondaryDataSource]] = Field(
575         default=None, description="Additional literature sources"
576     )
577
578     # Validators (10 total) defined via Pydantic decorators
579     # See Section S3 for implementation details

```

5.2 Input Model

Each input captures a single value extracted from literature with full provenance:


```

582 class InputType(str, Enum):
583     """Type of input extracted from literature."""
584     DIRECT_MEASUREMENT = "direct_measurement" # Value reported directly
585     PROXY_MEASUREMENT = "proxy_measurement" # Requires conversion
586     EXPERIMENTAL_CONDITION = "experimental_condition" # Protocol choice
587     INFERRED_ESTIMATE = "inferred_estimate" # Interpreted from qualitative text
588     ASSUMED_VALUE = "assumed_value" # Domain knowledge, not in source
589
590 class InputRole(str, Enum):
591     """Role of input in calibration."""
592     INITIAL_CONDITION = "initial_condition" # IC for ODE integration
593     TARGET = "target" # Calibration target (likelihood term)
594     FIXED_PARAMETER = "fixed_parameter" # Fixed value in model
595     AUXILIARY = "auxiliary" # Supporting data
596
597 class Input(BaseModel):
598     """A value extracted from literature with full provenance."""
599     name: str = Field(description="Unique identifier for this input")
600     value: float = Field(description="Extracted numeric value")
601     units: str = Field(description="Units of the value")
602     uncertainty: Optional[Uncertainty] = Field(default=None)
603     n: Optional[int] = Field(default=None, description="Sample size")
604     input_type: InputType
605     role: Optional[InputRole] = Field(default=None)
606     source_ref: str = Field(description="Reference to source_tag in data sources")
607     source_location: str = Field(description="Location within the source")
608     value_snippet: Optional[str] = Field(
609         default=None,
610         description="Exact text from paper containing the value (for validation)"
611     )

```

612 5.3 Model Type Discriminated Union

613 The schema uses Pydantic's discriminated union pattern to support multiple model types, each
614 with type-specific required fields:

```

615 class BaseForwardModelSpec(BaseModel):
616     """Base class for all forward model specifications."""
617     data_rationale: str = Field(
618         description="Why this model type fits the experimental data"
619     )
620     submodel_rationale: str = Field(
621         description="Why this is a valid submodel of the full QSP model"
622     )
623
624 class ExponentialGrowthModel(BaseForwardModelSpec):
625     """Exponential growth:  $dy/dt = k * y$ """
626     type: Literal["exponential_growth"] = "exponential_growth"

```

```

627     rate_constant: ParameterRole = Field(
628         description="Rate constant parameter name or input_ref"
629     )
630
631 class TwoStateModel(BaseForwardModelSpec):
632     """Two-state transition: A -> B with first-order kinetics."""
633     type: Literal["two_state"] = "two_state"
634     forward_rate: ParameterRole = Field(
635         description="Forward transition rate constant"
636     )
637
638 class CustomODEModel(BaseForwardModelSpec):
639     """Custom ODE with user-provided code."""
640     type: Literal["custom_ode"] = "custom_ode"
641     code: str = Field(description="Python ODE function")
642     code_julia: str = Field(description="Julia ODE function for inference")
643
644 # Discriminated union selects model class based on 'type' field
645 ForwardModel = Annotated[
646     Union[
647         FirstOrderDecayModel,
648         ExponentialGrowthModel,
649         LogisticModel,
650         MichaelisMentenModel,
651         TwoStateModel,
652         SaturationModel,
653         SteadyStateDensityModel,
654         SteadyStateFractionModel,
655         SteadyStateConcentrationModel,
656         SteadyStateRatioModel,
657         SteadyStateProliferationIndexModel,
658         BatchAccumulationModel,
659         AlgebraicModel,
660         DirectFitModel,
661         CustomODEModel,
662     ],
663     Field(discriminator="type"),
664 ]

```

The `discriminator="type"` annotation tells Pydantic to inspect the `type` field in the YAML to determine which model class to instantiate. This ensures that an `exponential_growth` model is validated against `ExponentialGrowthModel` (which requires `rate_constant`) while a `two_state` model is validated against `TwoStateModel` (which requires `forward_rate`).

5.4 Prior Specification

Priors are specified with distribution type and parameters:

```

671 class PriorDistribution(str, Enum):

```

```

672     """Supported prior distribution types."""
673     LOGNORMAL = "lognormal"      # For positive parameters (rates, densities)
674     NORMAL = "normal"            # For unconstrained parameters
675     UNIFORM = "uniform"          # For bounded parameters
676     HALF_NORMAL = "half_normal"  # For positive parameters with mode at 0
677
678 class Prior(BaseModel):
679     """Prior distribution specification for Bayesian inference."""
680     distribution: PriorDistribution
681     mu: Optional[float] = Field(default=None, description="Location parameter")
682     sigma: Optional[float] = Field(default=None, description="Scale parameter")
683     lower: Optional[float] = Field(default=None, description="Lower bound")
684     upper: Optional[float] = Field(default=None, description="Upper bound")
685     rationale: Optional[str] = Field(default=None)
686
687     @model_validator(mode="after")
688     def validate_prior_params(self) -> "Prior":
689         """Validate that required parameters are provided for each distribution."""
690         if self.distribution == PriorDistribution.LOGNORMAL:
691             if self.mu is None or self.sigma is None:
692                 raise ValueError("lognormal prior requires mu and sigma")
693             elif self.distribution == PriorDistribution.UNIFORM:
694                 if self.lower is None or self.upper is None:
695                     raise ValueError("uniform prior requires lower and upper")
696                 if self.lower >= self.upper:
697                     raise ValueError("uniform lower must be < upper")
698         return self

```

699 5.5 Loading and Validating YAML Files

700 To load and validate a target:

```

701 import yaml
702 from maple.core.calibration import SubmodelTarget
703
704 # Load YAML file
705 with open("psc_proliferation_PDAC_deriv001.yaml") as f:
706     data = yaml.safe_load(f)
707
708 # Parse and validate (raises ValidationError on failure)
709 target = SubmodelTarget(**data)
710
711 # Access validated fields
712 print(f"Target: {target.target_id}")
713 print(f"Parameters: {[p.name for p in target.calibration.parameters]}")
714 print(f"Model type: {target.calibration.model.type}")

```

715 If validation fails, Pydantic raises a `ValidationError` with detailed error messages:

```

716 pydantic_core._pydantic_core.ValidationError: 2 validation errors for SubmodelTarget
717 calibration.measurements.0.uses_inputs.0
718   Input 'typo_input_name' not found in inputs [...]
719 primary_data_source.doi
720   DOI '10.1234/fake' failed to resolve [...]

```

721 5.6 CalibrationTarget Model

722 The CalibrationTarget schema represents clinical and in vivo endpoints for full-model calibration.
 723 Where SubmodelTarget isolates a single mechanism, CalibrationTarget operates on the full model
 724 state.

```

725 class CalibrationTarget(BaseModel):
726     """
727     Calibration target for full-model Bayesian inference.
728
729     Represents a clinical or in vivo measurement that constrains
730     model behavior at the system level.
731     """
732     # Observable: how to compute the measurement from model species
733     observable: Observable = Field(
734         description="Maps full model species to measured quantity"
735     )
736
737     # Empirical data: literature-derived distribution
738     empirical_data: CalibrationTargetEstimates = Field(
739         description="Observed values derived from literature via Monte Carlo"
740     )
741
742     # Clinical/experimental context
743     experimental_context: ExperimentalContext = Field(
744         description="Species, system, indication, treatment, staging"
745     )
746     scenario: Optional[Scenario] = Field(
747         default=None,
748         description="Treatment scenario for intervention studies"
749     )
750
751     # Interpretation and provenance (shared with SubmodelTarget)
752     study_interpretation: str
753     key_assumptions: List[str]
754     key_study_limitations: Optional[List[str]] = None
755     primary_data_source: PrimaryDataSource
756     secondary_data_sources: Optional[List[SecondaryDataSource]] = None
757     source_relevance: Optional[SourceRelevanceAssessment] = None
758
759     # 30+ CalibrationTarget validators
760     # (categorized in Section S3 alongside SubmodelTarget validators)

```

5.7 Observable

The **Observable** specifies how to compute the experimental measurement from full model species. Unlike **SubmodelTarget**'s forward model (which defines isolated dynamics), the observable operates on the solved full-model state.

```
class Observable(BaseModel):
    """
    Full-model observable for CalibrationTarget.

    The code field contains a Python function that computes the
    measured quantity from model species at each time point.
    """
    code: str = Field(
        description="Python: compute_observable(time, species_dict, "
        "constants, ureg) -> Quantity array"
    )
    units: str = Field(description="Units of the observable output")
    species: List[str] = Field(
        description="Model species accessed (e.g., ['V_T.CD8', 'V_T.C1'])"
    )
    constants: Optional[List[ObservableConstant]] = Field(
        default=None,
        description="Named constants with provenance"
    )
    inputs: Optional[List[SubmodelInput]] = Field(
        default=None,
        description="Literature values used in the observable"
    )
    support: SupportType = Field(
        description="Mathematical domain: positive, non_negative, "
        "unit_interval, real"
    )
    mapping_rationale: str = Field(
        description="How the literature measurement maps to model species"
    )
    # Denominator audit fields for density/fraction observables
    experimental_denominator: Optional[str] = None
    model_denominator_species: Optional[List[str]] = None
```

Each **ObservableConstant** requires a `biological_basis` explanation and must trace to either the curated reference database or a specific literature source.

5.8 CalibrationTargetEstimates

The empirical data block derives summary statistics from literature inputs via Monte Carlo simulation.

```
class CalibrationTargetEstimates(BaseModel):
```

```

804     """
805     Empirical data block: literature values -> distribution for likelihood.
806     """
807     median: List[float] = Field(
808         description="Point estimates (length 1 for scalar, "
809         "or matching index_values for vector)"
810     )
811     ci95: List[List[float]] = Field(
812         description="95% credible intervals [[lo, hi], ...]"
813     )
814     units: str
815     sample_size: Union[int, List[int]]
816     sample_size_rationale: str
817
818     # For vector-valued data (time courses, dose responses)
819     index_values: Optional[List[float]] = None
820     index_unit: Optional[str] = None
821     index_type: Optional[IndexType] = None
822
823     # Inputs and assumptions
824     inputs: List[EstimateInput] = Field(
825         description="Literature values with provenance"
826     )
827     assumptions: Optional[List[ModelingAssumption]] = None
828
829     # Monte Carlo code
830     distribution_code: str = Field(
831         description="Python: derive_distribution(inputs, ureg) -> "
832         "{ 'median': [...], 'ci95': [...], 'units': str }"
833     )

```

834 5.9 Scenario and Intervention

835 For studies involving treatment, the `Scenario` captures the clinical context.

```

836 class Intervention(BaseModel):
837     """A single treatment intervention."""
838     agent: str = Field(description="Drug or treatment name")
839     dose: Optional[str] = None
840     route: Optional[str] = None
841     schedule: Optional[str] = None
842
843 class Scenario(BaseModel):
844     """Treatment scenario for intervention studies."""
845     interventions: List[Intervention]
846     treatment_line: Optional[str] = None
847     prior_treatments: Optional[List[str]] = None
848     description: str = Field(

```

```

849         description="Narrative description of the clinical scenario"
850     )

```

851 6 SubmodelTarget Schema Details

852 This section provides detailed field descriptions and representative YAML examples for each layer
853 of the SubmodelTarget schema. These examples are referenced from the Methods section; the
854 complete schema is defined in Section 5.

855 6.1 Input Fields

856 Each input captures a single value extracted from literature with full provenance. The key fields
857 are:

- 858 • **name**: Unique identifier referenced elsewhere in the schema
- 859 • **value** and **units**: The extracted numeric value with units
- 860 • **uncertainty**: Standard deviation or 95% confidence interval, when reported
- 861 • **n**: Sample size, for propagating uncertainty through the likelihood
- 862 • **input_type**: Classification as `direct_measurement`, `proxy_measurement`, `inferred_estimate`,
863 or `assumed_value`
- 864 • **role**: How the input is used: `target` (likelihood term), `auxiliary` (supporting), `initial_condition`,
865 or `fixed_parameter`
- 866 • **source_ref**: Reference to a literature source with DOI
- 867 • **value_snippet**: Quoted text from the source containing the value

868 Code block 1 shows a representative input.

```

inputs:
- name: pdgfr_fold_mean
  value: 4.37
  units: dimensionless
  uncertainty:
    sd: 0.49
  n: 3
  input_type: direct_measurement
  role: target
  source_ref: Schneider2001
  source_location: Abstract
  value_snippet: "Cell proliferation (4.37 +/- 0.49-
                  and 2.96 +/- 0.39-fold of control)."
```

Figure 1: An input capturing a proliferation measurement with provenance.

6.2 Parameter and Prior Specification

Each parameter includes a prior distribution with documented rationale. The schema supports four distribution families: LogNormal (positive parameters with multiplicative uncertainty), Normal (unconstrained), Uniform (bounded with no preference), and HalfNormal (positive with mode at zero).

Code block 2 shows a parameter specification.

```
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies  $k \sim \ln(4) \sim 1.4/\text{day}$ .
```

Figure 2: A parameter with lognormal prior and documented rationale.

6.3 Forward Model Specification

The `forward_model` field captures the dynamics relevant to the experiment. For time-course data, this is typically a simplified ODE; for steady-state or derived measurements, an algebraic model may suffice. State variables and the independent-variable block are nested inside `forward_model`.

Code block 3 shows a forward model using the exponential growth template.


```

calibration:
  forward_model:
    type: exponential_growth
    rate_constant: k_apsc_prolif
    independent_variable:
      name: time
      units: day
      span: [0.0, 1.0]
    state_variables:
      - name: aPSC_rel
        units: dimensionless
        initial_condition:
          value: 1.0
          rationale: Normalized to control baseline.
    data_rationale: |
      BrdU assay measures proliferation over ~24h with
      constant PDGF stimulus. Exponential growth fits.
    submodel_rationale: |
      At saturating PDGF, the full model reduces to
       $dN/dt = k_{apsc\_prolif} * N$ .

```

Figure 3: Forward model specification with state variable and time span.

6.4 Error Model Specification

The `error_model` field is a list of measurement entries, each specifying the statistical relationship between model predictions and one observed quantity. The `observation_code` derives the observed value and uncertainty from raw inputs. The `observable` defines the transformation from state variables to predicted measurement. The `likelihood` specifies the distribution family.

Code block 4 shows a complete error model entry.

```

error_model:
  - name: viability_48h
    units: dimensionless
    uses_inputs: [viability_mean, viability_sem]
    evaluation_points: [2.0] # days
    sample_size: 3
    observable:
      type: identity
      state_variables: [aPSC_norm]
    observation_code: |
      def derive_observation(inputs, sample_size):
          mean = inputs['viability_mean']
          sem = inputs['viability_sem']
          sd = sem * np.sqrt(sample_size) # SEM to SD
          return {'value': mean, 'sd': sd}
    likelihood:
      distribution: normal
      rationale: |
        Viability near 1 with small SD; normal adequate.

```

Figure 4: Error model specification linking forward model predictions to observed data.

7 Complete CalibrationTarget Example

This section presents a complete CalibrationTarget for baseline CD8+ T cell density in PDAC, demonstrating how full-model observables, reference database constants, and Monte Carlo distribution derivation work together.

7.1 Overview

This target constrains baseline intratumoral CD8+ T cell density using immunohistochemistry data from Jansen et al. (2021), who measured CD8 density across 599 pancreatic cancers using tissue microarrays, of whom 444 had both CD8 IHC and DOG1 staging information and contribute to the mixture-of-lognormals distribution used here. The observable computes CD8+ density from full model species (effector and exhausted CD8+ T cells) and reference database constants (cancer cell geometry). The distribution code combines two patient subgroups via a mixture of lognormals.

7.2 Observable

The observable maps full model species to the experimental measurement:

```

observable:
  code: |
    def compute_observable(time, species_dict, constants, ureg):
        import numpy as np
        cd8_eff = species_dict['V_T.CD8']
        cd8_exh = species_dict['V_T.CD8_exh']
        C1 = species_dict['V_T.C1']

```

```

906         cross_section = constants['pdac_cancer_cell_cross_section']
907         cellularity = constants['pdac_cellularity_fraction']
908
909         total_cd8 = cd8_eff + cd8_exh
910         tumor_area = C1 * cross_section / cellularity
911         density = total_cd8 / tumor_area
912         return density * ureg('cell / millimeter**2')
913     units: cell / millimeter**2
914     species: [V_T.CD8, V_T.CD8_exh, V_T.C1]
915     constants:
916         - name: pdac_cancer_cell_cross_section
917           value: 0.000227
918           units: millimeter**2 / cell
919           biological_basis: |
920             Mean cross-sectional area of PDAC cancer cells from
921             histomorphometric measurements. Derived from mean cell
922             diameter of 17 micrometers.
923           source_type: reference_db
924           reference_db_name: pdac_cancer_cell_cross_section
925         - name: pdac_cellularity_fraction
926           value: 0.15
927           units: dimensionless
928           biological_basis: |
929             Fraction of tumor volume occupied by cancer cells in PDAC.
930             PDAC tumors are characteristically desmoplastic with low
931             cellularity (10-20%).
932           source_type: reference_db
933           reference_db_name: pdac_cellularity_fraction
934     support: positive
935     mapping_rationale: |
936       IHC measures CD8+ cells per mm^2 of tissue section. The model
937       tracks CD8+ T cells as counts in the tumor compartment (V_T).
938       To convert model cell counts to density, we divide by tumor
939       area estimated from cancer cell count, cross-section, and
940       cellularity fraction.
941     experimental_denominator: "mm^2 of tumor tissue section area"
942     model_denominator_species: [V_T.C1]

```

943 7.3 Empirical Data

944 The distribution code combines two patient subgroups (DOG1-negative and DOG1-positive) via
 945 mixture-of-lognormals Monte Carlo:

```

946 empirical_data:
947     median: [138.8]
948     ci95: [[20.3, 965.2]]
949     units: cell / millimeter**2
950     sample_size: 444

```

```

951 sample_size_rationale: |
952     Total of 444 patients: 368 DOG1-negative + 76 DOG1-positive.
953 inputs:
954     - name: mean_cd8_stroma_neg
955       value: 227.7
956       units: cell / millimeter**2
957       source_ref: Jansen2021
958       source_location: "Table 4"
959       value_snippet: "CD8 ... 227.7 +/- 15.0"
960     - name: sem_cd8_stroma_neg
961       value: 15.0
962       units: cell / millimeter**2
963       source_ref: Jansen2021
964       source_location: "Table 4"
965       value_snippet: "CD8 ... 227.7 +/- 15.0"
966     - name: n_stroma_neg
967       value: 368
968       units: dimensionless
969       source_ref: Jansen2021
970       source_location: "Table 1"
971       value_snippet: "DOG1 negative 368 (61.4%)"
972     - name: mean_cd8_stroma_pos
973       value: 220.1
974       units: cell / millimeter**2
975       source_ref: Jansen2021
976       source_location: "Table 4"
977       value_snippet: "CD8 ... 220.1 +/- 33.0"
978     - name: sem_cd8_stroma_pos
979       value: 33.0
980       units: cell / millimeter**2
981       source_ref: Jansen2021
982       source_location: "Table 4"
983       value_snippet: "CD8 ... 220.1 +/- 33.0"
984     - name: n_stroma_pos
985       value: 76
986       units: dimensionless
987       source_ref: Jansen2021
988       source_location: "Table 1"
989       value_snippet: "DOG1 positive 76 (12.7%)"
990 distribution_code: |
991     def derive_distribution(inputs, ureg):
992         import numpy as np
993         rng = np.random.default_rng(42)
994         n_mc = 100000
995
996         # Subgroup 1: DOG1-negative
997         mean1 = inputs['mean_cd8_stroma_neg'].magnitude
998         sem1 = inputs['sem_cd8_stroma_neg'].magnitude

```

```

999         n1 = int(inputs['n_stroma_neg'].magnitude)
1000         sd1 = sem1 * np.sqrt(n1)
1001
1002         # Subgroup 2: DOG1-positive
1003         mean2 = inputs['mean_cd8_stroma_pos'].magnitude
1004         sem2 = inputs['sem_cd8_stroma_pos'].magnitude
1005         n2 = int(inputs['n_stroma_pos'].magnitude)
1006         sd2 = sem2 * np.sqrt(n2)
1007
1008         # Convert to lognormal parameters
1009         def to_lognormal(m, s):
1010             var = s**2
1011             mu = np.log(m**2 / np.sqrt(m**2 + var))
1012             sigma = np.sqrt(np.log(1 + var / m**2))
1013             return mu, sigma
1014
1015         mu1, sig1 = to_lognormal(mean1, sd1)
1016         mu2, sig2 = to_lognormal(mean2, sd2)
1017
1018         # Mixture sampling
1019         w1 = n1 / (n1 + n2)
1020         mask = rng.random(n_mc) < w1
1021         samples = np.where(
1022             mask,
1023             rng.lognormal(mu1, sig1, n_mc),
1024             rng.lognormal(mu2, sig2, n_mc)
1025         )
1026
1027         return {
1028             'median': [float(np.median(samples))],
1029             'ci95': [[float(np.percentile(samples, 2.5)),
1030                      float(np.percentile(samples, 97.5))],
1031             'units': 'cell / millimeter**2'
1032         }

```

1033 7.4 Source and Context

```

1034 primary_data_source:
1035     doi: "10.7717/peerj.11397"
1036     title: "Expression of immune cell markers in the tumor
1037            microenvironment and association with survival"
1038     authors: [Jansen, van Doorn, et al]
1039     year: 2021
1040     source_tag: Jansen2021
1041
1042 experimental_context:
1043     species: human
1044     system: clinical_resection

```

```

1045     indication: PDAC
1046     treatment_status: treatment_naive
1047     stage: resectable
1048
1049     study_interpretation: |
1050         Jansen et al. measured CD8+ T cell density across 599 pancreatic
1051         cancers using tissue microarrays and digital image analysis. We
1052         combine two subgroups (DOG1-positive and DOG1-negative) as a
1053         mixture of lognormals to capture the full population distribution.
1054
1055     key_assumptions:
1056         - "CD8 IHC detects both effector and exhausted T cells equally"
1057         - "Reported +/- values are SEM (confirmed by sqrt(n) test)"
1058         - "DOG1 expression does not significantly affect CD8 density"
1059
1060     key_study_limitations:
1061         - "TMA cores (0.6mm) may not represent whole-slide heterogeneity"
1062         - "Mixed pathologic stages without neoadjuvant therapy reporting"
1063         - "2D histology section vs. 3D model compartment"

```

1064 8 SubmodelTarget Source Characteristics

1065 This section provides detailed breakdowns of source quality, species translation, and indication
1066 match for the [37 curated](#) SubmodelTarget extractions.

1067 8.1 Source Quality

1068 Source quality categorizes the type of experimental evidence. *Human clinical* denotes data from hu-
1069 man patient samples or clinical measurements. *Human in vitro* denotes experiments using human-
1070 derived cell lines or primary human cells in culture. *Animal in vivo* denotes measurements from
1071 animal models (e.g., mouse xenografts). *Animal in vitro* denotes experiments using animal-derived
1072 cells in culture (e.g., murine splenocyte assays, rat stellate cell cultures).

Table 2: Distribution of primary data source quality across SubmodelTargets.

Source Quality	Count	%
primary_human_clinical	17	46%
primary_animal_in_vivo	9	24%
primary_animal_in_vitro	7	19%
primary_human_in_vitro	3	8%
review_article	1	3%
Total	37	

1073 8.2 Species Translation

1074 Species translation describes the mapping from the experimental organism to the model target
1075 species (human). Targets labeled human→human required no cross-species translation. Mixed-

species entries (e.g., human and rat→human) indicate that the extraction combined data from multiple species, such as a human clinical measurement anchored by rat-derived kinetic parameters.

Table 3: Species translation from data source to model target. Human→human indicates no cross-species translation required.

Translation	Count	%
human→human	21	57%
mouse→human	13	35%
rat→human	3	8%
Total	37	

8.3 Indication Match

Indication match describes how closely the experimental disease context matches the PDAC model target. *Exact*: data from PDAC patients or PDAC-derived models. *Proxy*: data from a different indication used as a mechanistic proxy (e.g., other cancer types, chronic pancreatitis, fibrosis models with similar stromal biology). *Related*: data from the same organ or disease class but not a direct match.

Table 4: How closely the data source indication matches the model target (PDAC).

Indication Match	Count	%
Data from PDAC patients/models	17	46%
Related indication (e.g., other cancers, fibrosis)	16	43%
related	4	11%
Total	37	

9 CalibrationTarget Detailed Metrics

This section provides detailed metrics for the 54 CalibrationTargets applied to the PDAC QSP model.

9.1 Observable Categories

9.2 Observable Complexity

The most complex observables (19 species) are neoadjuvant immunotherapy targets computing immune subset percentages of all nucleated cells, requiring all modeled immune, stromal, and tumor populations in the denominator. The highest-constant observables (6 constants) require geometric and cellularity reference parameters for converting between model cell counts and histologic measurements. Across all targets, observables reference 25 unique model species spanning immune cells (CD8⁺ effector and exhausted, Treg, Th, NK), myeloid cells (M1/M2 macrophages, MDSC, cDC1/cDC2), stromal cells (myCAF, iCAF, apCAF, quiescent PSC), tumor cells, ECM, and microenvironment variables (VEGF, lactate, pO₂, glucose, HIF).

Table 5: CalibrationTarget observable categories.

Observable Category	Count
Immune/stromal fractions	29
Cell densities	9
Population ratios	8
Metabolic/TME markers	3
Fold-changes	2
Clinical endpoints	2

Table 6: Observable complexity across 54 CalibrationTargets.

Metric	Mean	Min	Max
Model species per observable	7.4	1	19
Named constants per observable	0.6	0	6
Empirical inputs per target	5.7	1	18

9.3 Extraction Model Attribution

Table 7: CalibrationTarget extraction model breakdown.

Extraction Model	Count	Percentage
Claude Opus 4	27	50%
GPT-5.1	18	33%
Manually curated	9	16%
Total	54	

Of the 45 AI-generated targets, 30 (55%) are tagged as human-verified, indicating that a domain expert reviewed the complete target after initial extraction. 15 targets (27%) include data digitized from figures, where per-patient data points were manually extracted and the CalibrationTarget structure was assembled by the LLM from those data points.

9.4 Primary Data Sources

The 54 targets drew from 23 unique primary sources (by DOI), with publication years spanning 2000–2026. Two sources contributed the majority of targets: a neoadjuvant immunotherapy trial (Li et al., Cancer Cell, 2022) provided data for all 18 GVAX/nivolumab targets, and a large-scale immunohistochemistry study (Liudahl et al., Cancer Discovery, 2021) provided data for 12 baseline targets covering immune cell densities, fractions, and ratios.

10 Inference Results

All 37 SubmodelTargets were translated to a joint Julia inference script using Turing.jl¹. The translator identified 19 unique parameters and generated corresponding priors and likelihood terms. The plurality of targets (49%) use closed-form algebraic forward models; the remainder use structured model types including steady-state, batch accumulation, and first-order decay models (Table 8).

Table 8: Distribution of forward model types used in calibration submodels.

Model Type	Count	%
Algebraic (closed-form)	18	49%
Steady-state ratio	5	14%
Batch accumulation	4	11%
Steady-state density	4	11%
Steady-state fraction	3	8%
First-order decay ODE	2	5%
Steady-state concentration	1	3%
Total	37	

The generated script is self-contained, requiring only DifferentialEquations.jl, Turing.jl, and Distributions.jl². Each target contributes a simulate function, observed data constants, and a likelihood term. In this application, most targets constrain individual parameters through algebraic or structured forward models (steady-state, batch accumulation, first-order decay), with 11 parameters shared across multiple targets. The framework supports arbitrarily complex scenarios: multiple derivations constraining the same parameter, single targets constraining multiple parameters, and forward models involving ODEs that cannot be analytically inverted.

10.1 Convergence Diagnostics

We ran 4 chains of 1000 samples each using the NUTS sampler. All parameters achieved $\hat{R} < 1.01$, with bulk-ESS > 2600 and tail-ESS > 1300 throughout (Table 9). Sampling took about 42 seconds on an M-series MacBook Air.

10.2 Posterior Distributions

Figure 5 shows posterior estimates for all 19 parameters. Table 10 reports medians and 90% credible intervals.

Most posteriors are narrower than their priors, indicating that the data constrained the parameters. The tumor growth rate `k_C1_growth` ($5.34\text{e-}03 \text{ day}^{-1}$, 90% CI: $2.22\text{e-}03\text{--}0.0128$) implies a doubling time of about 130 days, consistent with clinical PDAC volume-doubling-time measurements[?] [?]. The activated PSC death rate `k_apsc_death` (0.0388 day^{-1}) implies a half-life of about 10 days.

The IL-2-dependent division counts for CD4+ T cells (8.03) fall within the 7–8 generation range expected from CFSE proliferation tracking, where dye dilution limits counting. The CD8+ estimate (5.31) falls below this range, suggesting either a genuinely lower IL-2 division capacity for the CD8+ subset in the source data or unmodelled differences between the CD4 and CD8 experimental conditions; the wide posterior for this parameter (see Table 10) reflects the limited identifiability from a single calibration target.

11 Model-Aware Prompt Construction Details

For each target, the prompt injects mechanistic context extracted from the model structure. For target parameters, the system automatically retrieves: the parameter’s name, units, and biological description; the reactions where the parameter appears with their rate laws; the species involved in

Table 9: Convergence diagnostics for MCMC sampling. All $\hat{R} < 1.01$ indicates convergence.

Parameter	$\hat{R}$	ESS (bulk)	ESS (tail)
N_IL2_CD4	1.000	4598	2549
N_IL2_CD8	1.002	2846	1334
cd8_exclusion_fraction	1.001	2705	2325
k_C1_growth	1.001	4912	3195
k_CCL2_deg	1.001	5256	2763
k_CCL2_sec	1.001	5453	2998
k_CD8_death	1.001	2851	2889
k_M1_pol	1.001	3020	2396
k_M2_pol	1.001	2773	2542
k_MDSC_death	1.001	2679	2515
k_MDSC_rec	1.001	2657	2621
k_Mac_death	1.000	3154	2987
k_Mac_rec	1.001	3055	2861
k_Treg_death	1.000	3813	2318
k_apsc_death	1.000	4222	2208
k_psc_activation	1.000	4511	3033
n_CD8_clones	1.001	4657	2895
q_CD8_T_in	1.000	4492	2440
q_Treg_T_in	1.002	5077	2879

Table 10: Posterior parameter estimates from joint Bayesian inference. Credible intervals are 90% highest density intervals. [†]Auxiliary parameter introduced during extraction.

Parameter	Median	90% CI	Units
N_IL2_CD4	8.03	[5.73, 10.3]	dimensionless
N_IL2_CD8	5.31	[4.03, 5.99]	dimensionless
cd8_exclusion_fraction	0.912	[0.741, 0.982]	dimensionless
k_C1_growth	5.34e-03	[2.22e-03, 0.0128]	day ⁻¹
k_CCL2_deg	4.29	[2.77, 6.75]	hour ⁻¹
k_CCL2_sec	3.32e-10	[2.57e-10, 4.33e-10]	nmol·cell ⁻¹ ·day ⁻¹
k_CD8_death	2.23	[0.546, 7.55]	day ⁻¹
k_M1_pol	0.0392	[0.0298, 0.0519]	day ⁻¹
k_M2_pol	0.0776	[0.0545, 0.111]	day ⁻¹
k_MDSC_death	0.08	[0.0568, 0.114]	day ⁻¹
k_MDSC_rec	1.36e+07	[9.76e+06, 1.92e+07]	cell·mL ⁻¹ ·day ⁻¹
k_Mac_death	0.0219	[5.42e-03, 0.0812]	day ⁻¹
k_Mac_rec	8.92e+05	[2.11e+05, 4.08e+06]	cell·mL ⁻¹ ·day ⁻¹
k_Treg_death	0.2	[0.0664, 0.636]	day ⁻¹
k_apsc_death	0.0388	[0.0278, 0.0506]	day ⁻¹
k_psc_activation	0.5	[0.342, 0.737]	day ⁻¹
n_CD8_clones	10.3	[8.66, 12.2]	dimensionless
q_CD8_T_in	4.03e-04	[3.52e-04, 4.60e-04]	min ⁻¹ ·cm ⁻³
q_Treg_T_in	6.47e-04	[4.63e-04, 9.05e-04]	min ⁻¹ ·cm ⁻³

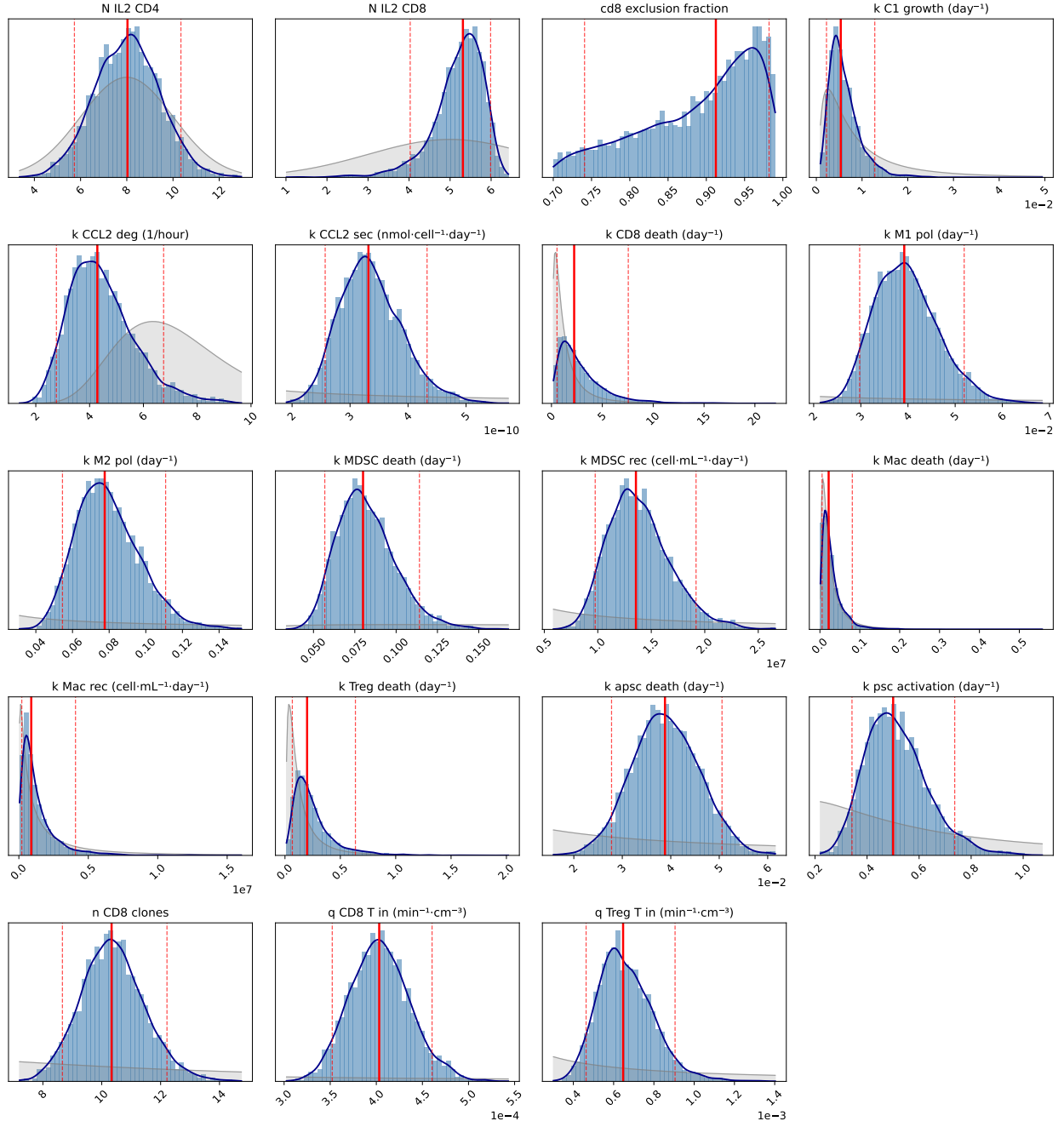


Figure 5: Posterior distributions from joint Bayesian inference. Gray shading shows the prior distribution; blue histogram and line show the posterior. Red lines indicate median (solid) and 90% credible interval bounds (dashed).

those reactions with descriptions; other parameters appearing in the same reactions; and the broader reaction network showing other reactions that involve the same species. For target observables, the system retrieves the observable’s name, units, and description, along with the species and compartments that define it. For example, when searching for data to calibrate a proliferation rate `k_apsc_prolif`, the prompt includes the reaction `aPSC -> 2*aPSC` with rate law `k_apsc_prolif * aPSC`, species descriptions explaining that `aPSC` represents activated pancreatic stellate cells, and related parameters like death rates that appear in the same reaction network.

This context guides the LLM’s search toward experiments that actually constrain the targets, rather than experiments that mention similar concepts but measure different quantities. For parameters, the broader reaction network helps the LLM identify whether additional parameters should be jointly calibrated. Extracting multiple parameters from the same study is intentional and preferred when the data supports it, as shared experimental conditions reduce inter-study variability.

To encourage literature diversity, source exclusion prevents the LLM from reusing papers across extractions. The prompt includes a list of primary sources already used for other targets, and a strict exclusion mode extends this to sources used by related parameters in the same reaction network.

Beyond mechanistic context, the prompt includes the complete schema structure with field descriptions, enabling the LLM to generate valid output without trial and error. The prompt also provides explicit guidance on common pitfalls: distinguishing SEM from SD (with conversion formulas), avoiding hardcoded numeric values in code blocks, and properly documenting source relevance for cross-species or cross-indication extractions. By showing validation requirements upfront (such as the prohibition on non-peer-reviewed sources and required uncertainty thresholds for proxy data), the prompt reduces the retry loop by preventing predictable errors.

12 Schema Implementation Details

Both schemas are implemented using Pydantic³, a Python library for declarative data validation. Each schema component is defined as a Pydantic model with typed fields and constraints. When YAML is parsed, Pydantic validates all constraints automatically: required fields must be present, values must match declared types, enumerations must use allowed values, and cross-field constraints must be satisfied.

The validation error messages are detailed and structured, reporting the field path, expected type, and actual value for each violation. This specificity is important for the LLM retry loop: when extraction fails validation, the error message provides actionable feedback. For example, a snippet validation error reports “Value-snippet mismatches (possible hallucination): input ‘cell_count_mean’ has value 4.37 but snippet contains ‘3.21’. Check that extracted values match the source text exactly.” This tells the LLM both what failed and how to fix it.

SubmodelTarget uses discriminated unions for its 15 forward model types: the `type` field determines which additional fields are required, ensuring type-specific validation without manual dispatch logic. CalibrationTarget uses a class hierarchy with shared base models for source documentation, relevance assessment, and experimental context.

13 Source Relevance Assessment Details

Each target includes structured metadata about its data sources. The `primary_data_source` block captures the main reference: DOI, title, authors, year, and a short `source_tag` used for cross-referencing elsewhere in the schema. Secondary sources that provide supporting context (e.g.,

reviews explaining biological mechanisms, or related studies used for uncertainty estimation) are listed in `secondary_data_sources`, each with a `contribution` field explaining how that source informs the extraction.

The `indication_match` field classifies how well the source disease context matches the model: `exact` (same indication), `related` (same organ or disease class), `proxy` (different tissue used as mechanistic proxy), or `unrelated`. Cross-indication extractions require a justification explaining why the data is still informative.

Species translation is captured by `species_source` and `species_target` fields. When these differ (e.g., mouse data informing a human model), the extraction must document the assumed translation.

The `source_quality` field categorizes the evidence type: `primary_human_clinical`, `primary_human_in_vitro`, `primary_animal_in_vivo`, `primary_animal_in_vitro`, `review_article`, or `textbook`. By default, non-peer-reviewed sources trigger a validation warning; in practice, preprints (e.g., from bioRxiv) that have been publicly available for several months can provide valuable data, and the validator can be configured to accept them with documented justification.

For immune and stromal parameters, `tme_compatibility` assesses whether the tumor microenvironment in the source matches the target model (`high`, `moderate`, or `low`). A T cell trafficking rate measured in melanoma (a T cell-permissive tumor) may not translate directly to PDAC (an immunosuppressive tumor); low compatibility requires explicit documentation.

Finally, `estimated_translation_uncertainty_fold` quantifies the expected uncertainty from all translation factors combined. Cross-species extraction from a proxy indication with low TME compatibility might warrant 10-fold or greater uncertainty, which propagates to prior width during inference.

14 Collaboration Mode Details

The two collaboration modes observed in this study offer different trade-offs. In the batch pipeline mode, the LLM produces a structured draft that the modeler then curates interactively: selecting appropriate forward model types and prior distributions for `SubmodelTargets`, correcting observable code and species mappings for `CalibrationTargets`, and revising source relevance assessments for both. This mode is efficient for initial coverage (many files per run) but carries a high rejection rate (58% in a documented batch review) because the LLM lacks real-time modeler guidance on source selection, species mapping, and denominator definitions. In the interactive mode, the modeler and LLM work together from the start, with the modeler directing extraction decisions while the LLM handles text generation, code writing, and literature comprehension. This mode produces near-final quality but requires deeper per-file engagement.

In both modes, the schema serves as the collaboration interface. It defines what information must be captured (ensuring completeness), constrains how it is represented (enabling validation), and separates data extraction from modeling decisions (clarifying where different types of expertise are needed). The validators provide guardrails during both LLM extraction and human editing, catching errors regardless of their source.

15 Detailed Comparison to Existing Approaches

General-purpose biomedical extraction frameworks like LLM-IE⁴ provide building blocks for named entity recognition and relation extraction, but do not address the downstream requirements of QSP calibration: uncertainty quantification, forward model specification, and code generation for

inference. Interactive systems like SciDaSynth⁵ support structured data extraction with human-in-the-loop refinement, but focus on tabular output rather than the hierarchical schema needed for QSP model calibration. QSP-Copilot⁶ applies LLMs to QSP model development broadly, including literature extraction, but emphasizes model structure discovery (entity-interaction pairs) rather than quantitative parameter calibration with provenance.

The validation approach in MAPLE draws on established strategies for hallucination detection. Requiring extracted values to appear in quoted source text is analogous to the grounding strategies used in retrieval-augmented generation⁷, where LLM outputs are verified against retrieved documents. The schema-based validation enforces structural constraints similar to those in constrained decoding and structured output systems, where JSON schemas guide LLM generation toward valid output⁸. The key contribution is combining these strategies with domain-specific validators (DOI resolution, unit parsing, code execution) in a framework tailored to quantitative scientific extraction.

Recent work on knowledge graph construction from biomedical literature^{9,10} demonstrates that LLMs can extract structured relationships at scale. The Immune Cell Knowledge Graph (ICKG) used zero-shot prompting with Llama 3.1 to extract activation and inhibition relationships from over 24,000 PubMed abstracts. Such approaches prioritize coverage and can tolerate some extraction errors because downstream analyses aggregate across many relationships. Parameter calibration has different requirements: each extracted value directly influences model predictions, so individual errors matter more. MAPLE trades some throughput for higher per-target validation stringency.

The IQ Consortium has outlined best practices for AI/ML in quantitative modeling, including the use of NLP for knowledge extraction from biomedical literature¹¹. They emphasize explainability, human-in-the-loop oversight, and consideration of context of use. MAPLE aligns with these principles: the schema documents reasoning and provenance, validators catch errors before human review, and the generated code is transparent and modifiable. The growing interest in coupling QSP with machine learning^{12,13,14,15} suggests that hybrid approaches combining mechanistic modeling with data-driven methods will become increasingly important; automated literature extraction is one component of this broader integration. Recent work has also explored using LLMs directly as tools for comparing and synthesizing perspectives on QSP methodology¹⁶. While we evaluate MAPLE on a QSP model, the approach is not limited to pharmacology: any parameter-rich mechanistic model that draws calibration data from literature (e.g., systems biology signaling networks, physiologically-based models) could benefit from structured extraction with validation.

References

- [1] Ge H, Xu K, Ghahramani Z. Turing: A Language for Flexible Probabilistic Inference. In: Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics. PMLR; 2018. p. 1682-90. Available from: <https://proceedings.mlr.press/v84/ge18b.html>.
- [2] Besançon M, Papamarkou T, Anthoff D, Arslan A, Byrne S, Lin D, et al. Distributions.jl: Definition and Modeling of Probability Distributions in the JuliaStats Ecosystem. Journal of Statistical Software. 2021;98(16):1-30. Available from: <https://www.jstatsoft.org/v098/i16>. doi:10.18637/jss.v098.i16.
- [3] Colvin S, Jolibois E, Ramezani H, Garcia Badaracco A, Dorsey T, Montague D, et al.. Pydantic Validation; 2025. Available from: <https://github.com/pydantic/pydantic>.

- [4] Hsu E, Roberts K. LLM-IE: A Python Package for Biomedical Generative Information Extraction with Large Language Models. *JAMIA Open*. 2025 Apr;8(2):ooaf012. doi:10.1093/jamiaopen/ooaf012.
- [5] Wang X, Huey SL, Sheng R, Mehta S, Wang F. SciDaSynth: Interactive Structured Data Extraction From Scientific Literature With Large Language Model. *Campbell Systematic Reviews*. 2025;21(4):e70073. doi:10.1002/cl2.70073.
- [6] Saini A, Farnoud A. QSP-Copilot: An AI-Augmented Platform for Accelerating Quantitative Systems Pharmacology Model Development. *CPT: Pharmacometrics & Systems Pharmacology*. 2025;14(11):1775-86. doi:10.1002/psp4.70127.
- [7] Li Y, Fu X, Verma G, Buitelaar P, Liu M. Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems. *arXiv*; 2025. doi:10.48550/arXiv.2510.24476.
- [8] Geng S, Cooper H, Moskal M, Jenkins S, Berman J, Ranchin N, et al. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. *arXiv preprint arXiv:250110868*. 2025 Feb. doi:10.48550/arXiv.2501.10868.
- [9] He S, Tan Y, Ye Q, M Gubin M, Dede M, Rafei H, et al. AI-powered Immune Cell Knowledge Graph (ICKG) with Granular Immune Contexts Enables Immune Program Interpretation. *npj Artificial Intelligence*. 2026 Jan;2(1):13. doi:10.1038/s44387-025-00060-4.
- [10] Gao T, Zhai X, Yang C, Lv L, Wang H. Joint Extraction of Entity and Relation Based on Fine-Tuning BERT for Long Biomedical Literatures. *Bioinformatics Advances*. 2024 Jan;4(1):vbae194. doi:10.1093/bioadv/vbae194.
- [11] Terranova N, Renard D, Shahin MH, Menon S, Cao Y, Hop CECA, et al. Artificial Intelligence for Quantitative Modeling in Drug Discovery and Development: An Innovation and Quality Consortium Perspective on Use Cases and Best Practices. *Clinical Pharmacology & Therapeutics*. 2024;115(4):658-72. doi:10.1002/cpt.3053.
- [12] Folguera-Blasco N, Boshier FAT, Uatay A, Pichardo-Almarza C, Lai M, Biasetti J, et al. Coupling Quantitative Systems Pharmacology Modelling to Machine Learning and Artificial Intelligence for Drug Development: Its pAIns and gAIns. *Frontiers in Systems Biology*. 2024 Jul;4. doi:10.3389/fsysb.2024.1380685.
- [13] Pawłowski T, Bokota G, Lazarou G, Kierzek AM, Sroka J. Emulation of Quantitative Systems Pharmacology Models to Accelerate Virtual Population Inference in Immuno-Oncology. *Methods (San Diego, Calif)*. 2024 Mar;223:118-26. doi:10.1016/j.ymeth.2023.12.006.
- [14] Goryanin I, Goryanin I, Demin O. Revolutionizing Drug Discovery: Integrating Artificial Intelligence with Quantitative Systems Pharmacology. *Drug Discovery Today*. 2025 Sep;30(9):104448. doi:10.1016/j.drudis.2025.104448.
- [15] Androulakis IP, Cucurull-Sanchez L, Kondic A, Mehta K, Pichardo C, Pryor M, et al. The Dawn of a New Era: Can Machine Learning and Large Language Models Reshape QSP Modeling? *Journal of Pharmacokinetics and Pharmacodynamics*. 2025 Jun;52(4):36. doi:10.1007/s10928-025-09984-5.

- 1310 [16] Androulakis IP, Cheng L, Cho CR, Zhang T. Leveraging Large Language Models to Compare
1311 Perspectives on Integrating QSP and AI/ML. *Journal of Pharmacokinetics and Pharmacody-*
1312 *namics*. 2025 May;52(3):29. doi:10.1007/s10928-025-09976-5.